

Welcome to the RocketRide Buildathon!

Step 1 · Understanding the Challenge

Step 2 · Inspiration: App Ideas for You

Step 3 · Learning the Tools

Part 1 · Your Mission

Build Something Real

Your goal is to create a complete AI application that solves a real problem. Think beyond just a "demo" or a simple script—aim for something a user would actually pay for!

What is RocketRide?

Think of RocketRide as a **smart workspace** (or "meta-harness") for AI. It helps you connect different AI models together into a **pipeline**—which is just a sequence of steps that process data automatically.

Why use RocketRide?

- **Save Time:** We handle the infrastructure so you can focus on your code.
- **Scale Easily:** Move from one test to thousands of files without breaking a sweat.
- **Collaborate:** Work with your team on shared AI workflows in the cloud.

How to Win: Make it "Load-Bearing"

A "load-bearing" app is one where RocketRide is doing the heavy lifting. Don't just make one AI call; build a system that uses these features:

- **Batch Processing:** Handle thousands of records at once.
- **Mixed Media:** Use text, images, and audio in one single flow.
- **Multi-Agent:** Use "specialist" AIs that check each other's work.

What a Winning Submission Looks Like

- **A Specific User:** Solve a problem for a specific group, like "student study planners."
- **Automatic Work:** Input should arrive via uploads or feeds, not just copy-pasting.
- **Real-World Action:** The app should send an email, file a ticket, or update a database.
- **Human-in-the-loop:** The AI should ask a person for help if it isn't confident.

Judging Criteria

We are looking for apps that are publishable, solve a real problem, and handle errors gracefully. Avoid submissions that only work on a few samples or don't have a human oversight step.

What we are judging

	What wins
Is it a real app?	Publishable. A stranger could sign up and get value without you explaining it.
Would someone pay?	A named user with a named problem, not a category.
Is RocketRide the engine?	Pipelines do the work. Not one model call behind a landing page.
Does it hold up at volume?	We hand it a batch, not a demo input. It holds, and the cost is sane.
Does it know when to stop?	A validator, a confidence line, a human gate.
Would it still run next month?	Malformed input, provider timeouts, a predictable cost per run.

Disqualifies: works on three sample inputs only · no path to a human anywhere · no idea what it costs per run · a single model call with a landing page.

Part 2 · Choose your Million dollar App Idea to Build from or come up with something

Every one is a full agentic system: it ingests continuously, keeps memory that compounds, takes real action with a human gate where the stakes demand it, and has a buyer with an existing budget line.

Judging bar: load-bearing use. A one-line SDK import that is never called again does not count.

2. Bid Factory

For B2B companies that respond to RFPs and tenders with dedicated teams. Monitors procurement portals and incoming RFPs, and keeps a growing answer library - every past Q&A, win/loss result, pricing call, and compliance document - all searchable. Reads each new RFP, matches requirements to existing answers, writes what's missing, builds the full response with a compliance checklist, and schedules reviews - with multiple tools working together and humans approving each section and the final submission. Revenue: per-seat subscription (high contract value); each RFP is a large, billable workload.

3. Churn Rescue Desk

For SaaS companies trying to keep their revenue from shrinking. Constantly reads product usage, billing activity, support tone, and when key contacts change jobs; tracks what healthy vs. churned accounts looked like at each stage and which actions saved which type of customer. Scores churn risk continuously, picks the right response playbook, and runs it - writes the outreach email, creates a task for the customer success manager, applies pre-approved discounts within set limits - humans approve anything above a certain threshold, and every result improves the playbook. Revenue: % of saved recurring revenue or a monthly per-account fee; always-on scoring means constant compute usage.

4. AP Payment Fraud Sentinel

For mid-size finance teams paying hundreds of vendors, blocking email fraud and fake bank-detail changes. Checks every invoice, vendor detail update, and approval against a database of vendors, bank accounts, approvers, and past payment behavior - the patterns fraud hides inside. Automatically holds anything suspicious and verifies it out-of-band, including calling the vendor on their known number rather than any number in a suspicious email; a human must manually release every held payment. Revenue: fee per invoice screened plus % of losses prevented; one blocked six-figure fraud pays for years of the product.

5. Compliance Change Radar

For compliance teams at banks, fintechs, and insurers working across multiple countries. Watches regulator announcements and maintains the client's compliance register as a connected map - each regulation linked to the internal policies, controls, and owners that implement it. Every new regulatory update is compared against that map to find exactly which policies are affected; the system drafts the required change, creates a ticket for the responsible

owner, and keeps the audit trail regulators will want - compliance officer approves every change. Revenue: per-country subscription against one of B2B's most stable budget lines; runs on a regular schedule.

6. Field Claims Copilot

For insurance companies and claims processors handling property and car damage. Takes in claim submissions, field photos and videos, and weather/disaster data; maintains a record linking policy details to claim history, known fraud patterns, and local repair cost benchmarks. Assesses damage severity from all the evidence, generates a repair estimate, books inspections via calendar and automated phone calls, flags potential fraud, and prepares the settlement recommendation - a licensed human adjuster approves every settlement. Revenue: priced per claim; photos, voice, and documents together make this a heavy, well-monetized workload.

7. Warranty & Returns Arbiter

For electronics and appliance brands overwhelmed by return and warranty requests. Takes in return requests, device diagnostic data, customer photos/videos of faults, and retailer data; builds a product-wide fault map showing which parts fail, in which production batches, under which conditions, alongside each customer's purchase and claim history. Decides repair/replace/refund/deny with supporting evidence, issues return labels and authorizations, sends all denials to human review, and spots patterns early ("this batch's hinge failure rate just tripled") before they turn into recalls. Revenue: fee per return processed plus an upsell for fault intelligence; constant device data ingestion is steady volume.

1. Chargeback Defender

For online stores and SaaS businesses losing money to payment disputes. Tracks dispute alerts, orders, user sessions, and delivery data; builds a per-customer record linking all of these plus past dispute results. When a new dispute comes in, it gathers evidence from all systems, packages it in the format the payment processor needs, and submits it before the deadline - a human checks before every submission - then records the result so future wins and losses improve the process. Revenue: % of money recovered or a fee per dispute; every dispute is a paid job.

8. Media Rights Sentinel

For content owners - studios, labels, course creators, sports bodies - losing licensing money to unauthorized use. Scans new uploads, marketplace listings, and broadcast feeds; keeps a content fingerprint database linked to license agreements, regions, and enforcement history, so a legitimate licensee is never confused with a pirate. Matches what it finds against the license database, estimates the value of each violation, and takes action - takedown, licensing offer, or legal escalation pack - with a human approving every outbound claim. Revenue: % of licensing revenue recovered; fingerprint matching at scale shows off the system's parallel processing power.

9. Global Workforce Compliance Guard

For companies hiring contractors and remote workers across many countries. Reads timesheets, invoices, contracts, and employment law updates per country; maintains a connected record of each worker, their contract, their country, and their classification, with full history. Continuously re-checks misclassification risk as laws and work patterns shift, drafts contract updates, keeps the filing calendar per country, and builds the audit-defense file - country-specific agents coordinating under a human legal gate on every change. Revenue: per-worker monthly fee against a serious risk (misclassification fines); scheduled re-checks are recurring compute.

10. Ad Spend Autopilot

For performance marketing teams managing significant monthly budgets across channels. Tracks live campaign results, pacing, and how each creative is performing; keeps a record of which message, format, and audience combination worked - so that knowledge survives team changes and platform switches. Shifts budget within set limits, pauses underperforming ads, writes the next creative brief based on proven results, and sends the weekly attribution report - with analysts, a budget tool, and a brand-safety reviewer involved, plus human approval above spend-change limits. Revenue: % of managed spend or tiered budget fees; always-on optimization means constant usage.

11. Denial Recovery Desk

For hospital and clinic billing teams losing 3-5% of revenue to insurance denials. Reads payment files, clinical notes, prior authorization records, and insurance policy updates; keeps a record of which argument, with which clinical evidence, actually overturned which denial code at which insurer. For each denial it builds the appeal with exactly the right supporting documents,

files before the insurer's deadline, and escalates to a doctor-to-doctor review when the data shows written appeals won't win - a coder or clinician approves every clinical claim. Revenue: % of recovered revenue, the standard model in this space; every denial is a billable job and denial volume never drops.

12. Deduction Defender

For consumer brand finance teams losing 1-2% of revenue to retailer chargebacks and unauthorized deductions. Reads payment remittances, promotion agreements, shipment data, and retailer compliance manuals; builds a record that learns which retailer takes which invalid deduction and which dispute package gets money returned. Checks every deduction against the agreement that supposedly authorized it, auto-disputes invalid ones with proof attached, and writes off valid ones without wasting human time - deductions above a set amount need analyst approval. Revenue: % of recovered deductions plus a per-retailer subscription; every remittance file is heavy recurring volume.

13. Change Order Claims Desk

For contractors and subcontractors who lose margin because delay and change claims are filed late or without solid evidence. Reads daily site reports, RFIs, submittals, schedule updates, weather data, and site photos; builds a connected record linking every disruption to the contract clause that allows recovery and the evidence that proves it. When a delay happens it drafts the notice within the contractual deadline - where most claims die - builds the entitlement case with dated evidence, and prices the impact - the project manager approves every submission. Revenue: % of recovered claim value or a per-project fee; constant site document ingestion across a portfolio is sustained compute.

14. Lease Audit Engine

For retail, restaurant, and enterprise tenants overpaying landlords on operating expense bills that nobody has time to review. Reads lease documents, annual expense reconciliation statements, landlord invoices, and property tax records; keeps a per-site record linking every clause, obligation, and billed amount - so an exclusion agreed in a 2019 amendment still gets enforced in the 2026 bill. Recalculates what should have been charged, flags overcharges with the exact clause cited, drafts the dispute letter, and tracks recovery to close - a real estate lead approves every letter to the landlord. Revenue: % of recovered overcharges, the standard model for lease audit firms; every reconciliation cycle across a portfolio is a large batch job.

15. Revenue Leakage Guard

For B2B SaaS finance and RevOps teams quietly under-billing because contracts and billing setup drift apart over time. Reads signed contracts, billing system config, entitlement records, and product usage data; maintains a record tracking what each customer agreed to, what they were given access to, what they actually used, and what they were charged. Continuously compares all four, raises catch-up billing proposals where usage exceeded entitlements, flags things provisioned but never billed, and catches expired discounts still being applied - finance approves every customer-facing adjustment. Revenue: % of recovered leakage or a subscription tiered by annual revenue; continuous reconciliation means always-on usage.

16. Grant Compliance Manager

For universities, hospitals, and nonprofits managing restricted funding where one disallowed expense becomes a repayment demand. Reads grant agreements, expense and payroll data, effort certifications, and funder policy updates; keeps a record of every grant, its restrictions, every expense charged to it, and whether it was allowed - with full history so the same cost type is never judged differently across grants. Checks every transaction against the specific grant's rules, blocks or flags disallowed spend before it posts, builds financial and progress reports by funder deadlines, and prepares the audit-defense file - the grants officer approves every determination. Revenue: per-grant monthly subscription against a serious audit risk; constant transaction screening plus scheduled reporting.

17. Utility Spend Auditor

For businesses running many sites - retail chains, manufacturers, logistics networks - quietly paying the wrong electricity or gas rate across dozens of meters. Reads utility bills across suppliers and formats, meter interval data, published rate schedules, and contract terms; keeps a per-site record of every meter, its correct rate, its usage, and every past claim outcome. Recalculates each bill against the rate the site should actually be on, finds billing errors, overcharges, demand-charge savings, and unbilled or double-billed meters - then files refund claims with the utility and tracks them to credit, with facilities approving anything that changes a contract. Revenue: % of savings and recovered credits; monthly bill processing across hundreds of meters is steady batch volume.

18. Franchise Standards Auditor

For franchise brands whose reputation depends on hundreds of independently run locations they can only physically visit twice a year. Reads location-submitted photos and videos, inspection reports, review and complaint feeds, and point-of-sale signals; keeps a record per location of every standard, every violation, and every fix - showing which issues keep coming back, which get fixed fast, and which signal a franchisee heading toward termination. Scores each location continuously, schedules re-inspections based on risk rather than a fixed calendar, sends cure notices with evidence attached, and escalates repeat violations into a formal default package - a franchise manager approves every notice. Revenue: per-location monthly subscription; constant photo and video processing across a network is exactly the heavy workload this system monetizes best.

19. IP Docket Guard

For in-house IP lawyers and patent firms where one missed deadline permanently kills an asset worth more than the firm. Reads patent office updates across countries, docketing records, renewal fee schedules, and internal product portfolio data; maintains a connected record of every patent family, country, deadline, and business value - so renewal decisions are made based on what the asset is actually worth, not just by default. Tracks every legal deadline, drafts office action responses using the patent's own filing history, and escalates renewal decisions with a business case attached rather than silently paying or letting patents lapse - attorney sign-off required on every filing. Revenue: per-asset annual subscription against catastrophic risk; portfolio-wide monitoring runs continuously.

20. Safety Reporting Command

For EHS managers at industrial and construction sites where filing a regulatory report late is itself a violation. Reads incident and near-miss reports including field voice notes, inspection findings, equipment sensor data, and corrective action records; keeps a connected record of every hazard, control, incident, and fix across all sites - so a near-miss pattern at one plant automatically flags the same unguarded hazard at eleven others. Classifies each incident against reporting thresholds, files the regulatory report within the legal window, assigns and follows up on corrective actions until closed, and surfaces which sites are trending toward a serious event - the EHS lead approves every regulatory filing. Revenue: per-site monthly subscription; constant incident data ingestion plus scheduled reporting.

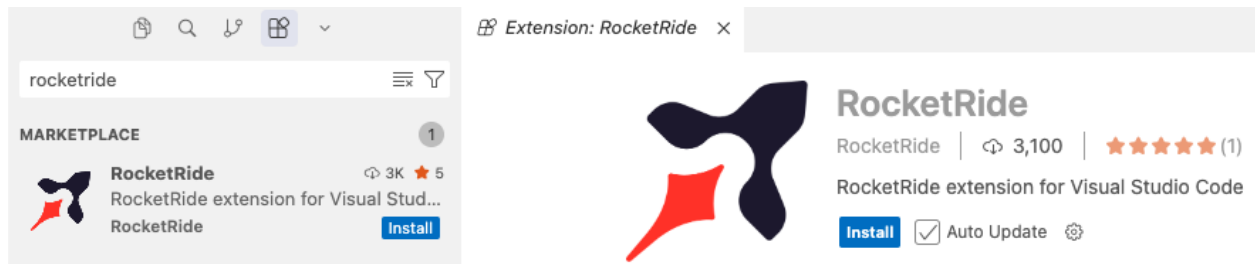
Part 3 · How to use RocketRide

1. Get set up

None of the prep needs a promo code. Local mode is free and works today, so do it at home.

Before the day, at home (about 45 minutes, mostly downloads)

1. **Join the Discord:** <https://discord.gg/PMXrtenMsY>
2. **Install the extension.** [VS Code Marketplace](#), search "RocketRide". On **Cursor**, **Windsurf** or **VSCodium** use [Open VSX](#) instead, they cannot reach the Microsoft marketplace



3. **Connect in Local mode** and let the engine download (section 5). Free, no account needed
4. **Build and run one small pipeline** so you have seen it work
5. **Sort out a model.** Your own OpenAI or Anthropic key, or install [Ollama](#) and `ollama pull llama3.2`

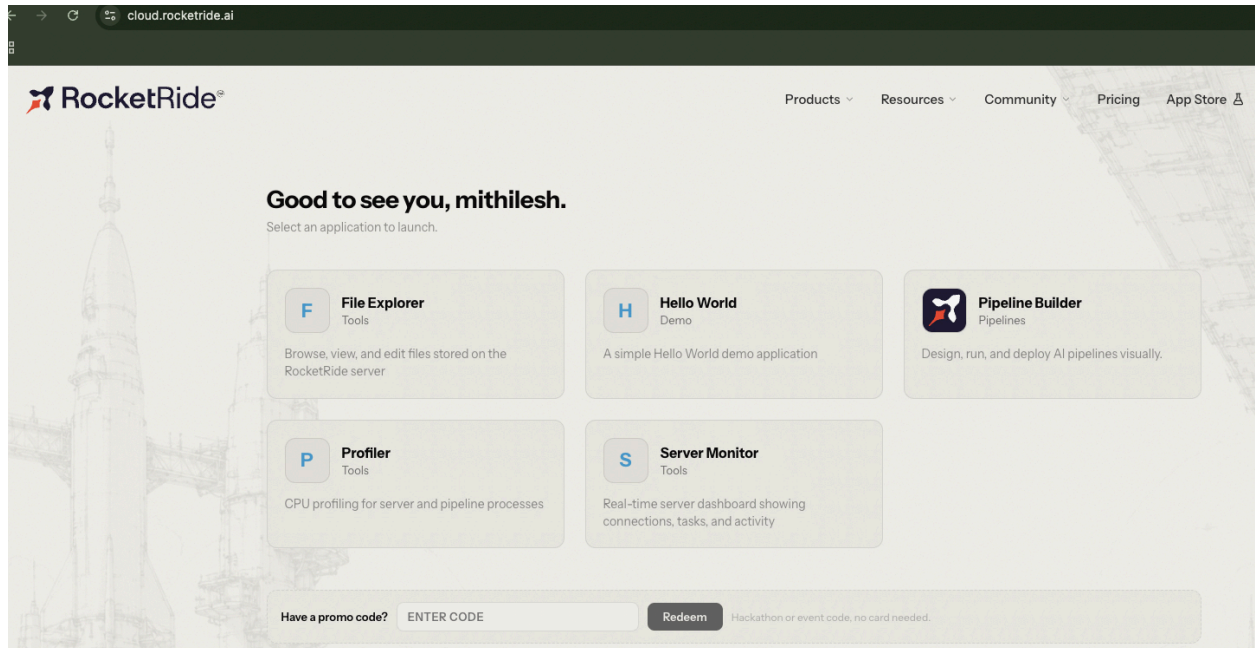
You need: VS Code or a fork of it, a few GB of disk, and Python 3.10+ only if you want the SDK in section 6. No Docker, no account, no promo code to build locally.

Do the downloads at home. Three hundred people pulling engine binaries and model weights at once on venue wifi is the most predictable way to lose the first hour.

On the day, at the venue

6. **Sign up at** cloud.rocketride.ai
7. **Redeem the promo code** we hand out. Bottom of the app launcher, **Have a promo code?**, no card needed
8. **Switch the extension to Cloud** when you are ready to deploy and demo (section 5)

Until you redeem, **Pipeline Builder** is badged **Payment Required**. Once you do, it unlocks and your credits are live.



What the credits cover, and what they do not

This is what stalls people. Your promo code covers **RocketRide only**. It does **not** pay for model API calls. OpenAI, Anthropic and the rest bill you separately.

So bring your own key, share a teammate's, or use **Ollama**, which runs models on your machine for nothing and keeps working when the venue wifi does not.

Stuck at any point: ask **Rocket Ralph** in [#support](#). It answers at any hour. Rocketride Team members are in the channel during working hours in India and California.

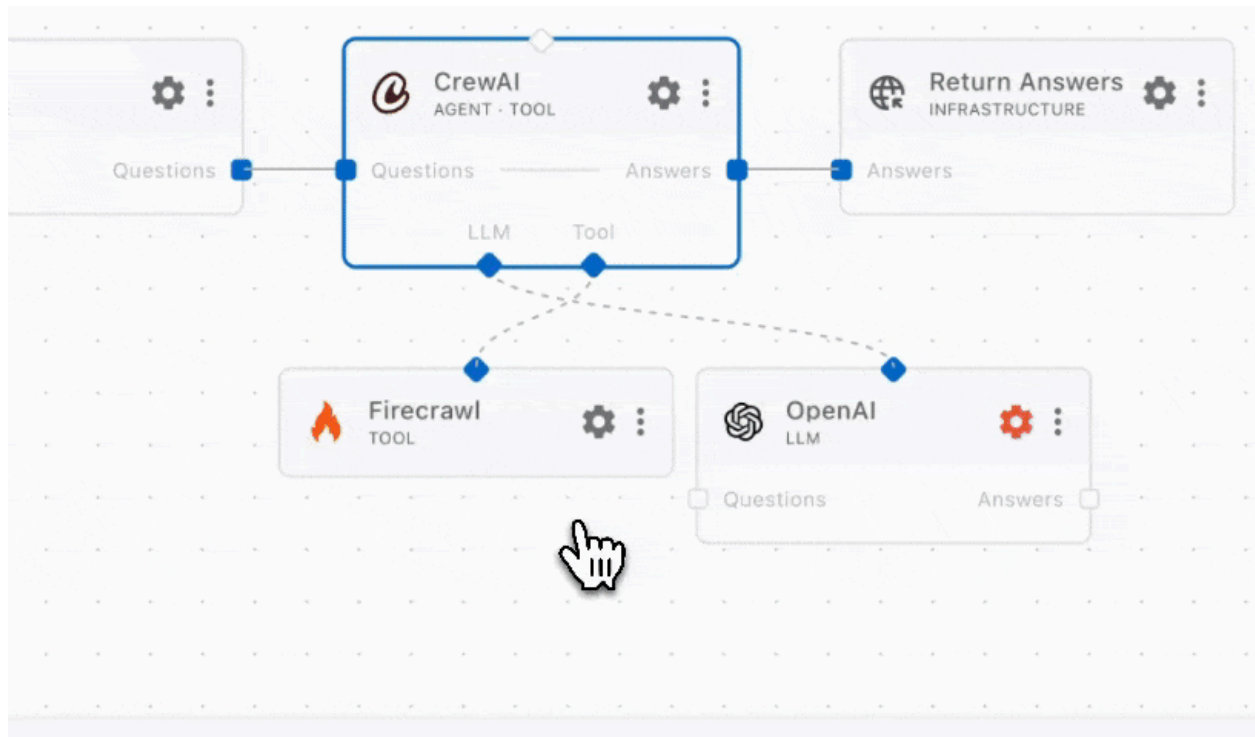
2. Three places to build

All three produce the same thing, a **.pipe** file, and it runs anywhere. **Pick one and stay there for your first build.**

	How
Canvas in VS Code	Drag and drop
Pipeline Builder in Cloud	Drag and drop in the browser
A coding agent	Claude, Cursor or Windsurf writes the JSON

The canvas

Watch this first, it takes a minute:



https://raw.githubusercontent.com/rocketride-org/rocketride-server/develop/images/first_pipe.gif

Make a new file ending in `.pipe` and it opens straight into the canvas. You get **Select your starting point: Chat** to type at it, **Dropper** to drop files in, **Web Hook** for something else calling you. It also offers templates that already run, including **RAG Chat** and **Document Parser**. Starting from a template and deleting what you do not need is usually faster than building from scratch.

Drag nodes on, connect the sockets, hit **Play**. A **red gear means a node is not configured**, usually a missing API key. While it runs you get live traces: execution path, token usage, memory stats. That is where you look when something misbehaves, because it shows you which node stopped rather than making you guess.

Build one node at a time and run after each addition. Wiring five nodes and then debugging is far slower.

3. The one concept worth understanding

There are **two kinds of wire** on the canvas and they mean different things. This trips up everybody once.

	Data lanes	Invoke connections
Look	Solid line, square sockets	Dashed line, diamond sockets

Position	Left and right edges	Bottom edge
Labels	Questions, Answers, text, data	LLM, Tool, Memory
Means	Data flows from here to there	This node can <i>call</i> that node
In JSON	<code>input</code> array	<code>control</code> array

Data lanes are the pipeline itself, left to right, in order.

Invoke connections are capabilities you hand to a node. An agent does not pass data "into" a model in sequence, it *calls* the model whenever it needs to think, as many times as it likes.

Same with a tool: the agent calls it when it decides it needs a web page.

That is why they hang underneath rather than sitting in the flow.

A lane is a pipe. An invoke connection is a phone number.

4. The `.pipe` file format

JSON. The key you always need is **components**, the array of nodes. Each has an `id`, a `provider` (the node type, like `webhook` or `llm_openai`), a `config`, and optionally `input` (data connections in) and `control` (invoke connections in).

This is exactly the canvas, written down. `input` is the solid lines, `control` is the dashed ones.

A webhook straight to a response, the smallest thing that runs:

```
JSON
{
  "components": [
    { "id": "in", "provider": "webhook", "config": {} },
    {
      "id": "out",
      "provider": "response",
      "config": { "laneName": "text" },
      "input": [{ "lane": "text", "from": "in" }]
    }
  ]
}
```

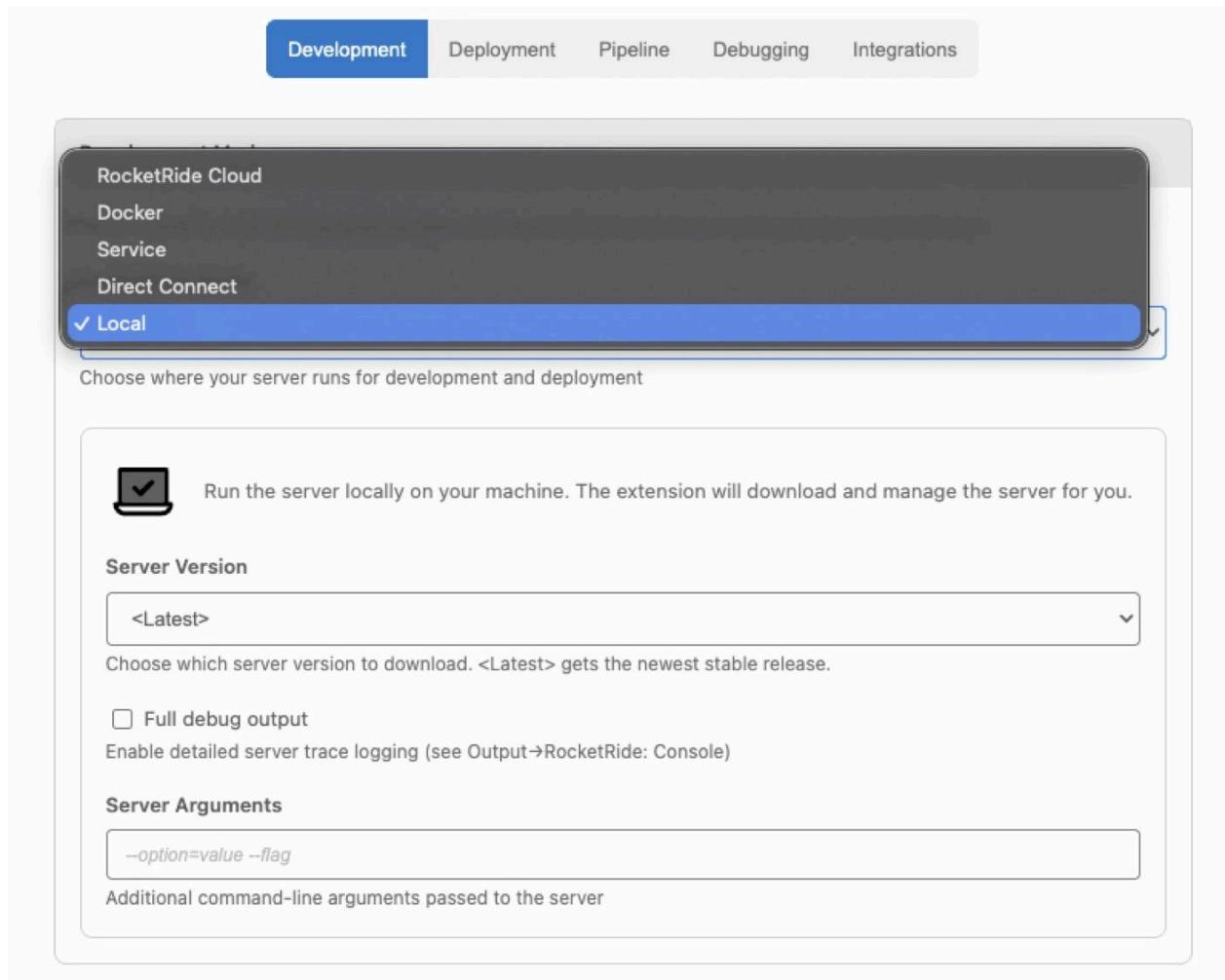
5. Local or Cloud

The same **.pipe** file runs in both places, unchanged. You are changing one dropdown. In VS Code, open the **RocketRide extension settings**, **Development** tab, **Connection mode**.

Option 1: Local

The extension **downloads and manages the engine for you**. Leave Server Version on **<Latest>**. Turn on **Full debug output** when stuck, then read **Output → RocketRide: Console**.

Connecting locally also **writes `ROCKETRIDE_URI` and `ROCKETRIDE_APIKEY` into your `.env`**, which you need in section 6. So connect through the extension once before writing any code.



The screenshot shows the VS Code interface with the 'Development' tab selected in the top navigation bar. Below the navigation bar, a dropdown menu is open, showing the following options: 'RocketRide Cloud', 'Docker', 'Service', 'Direct Connect', and 'Local'. The 'Local' option is selected and highlighted with a blue bar. Below the dropdown, the text 'Choose where your server runs for development and deployment' is visible. The main settings area contains a section for 'Local' configuration, which includes a description: 'Run the server locally on your machine. The extension will download and manage the server for you.' Below this, there is a 'Server Version' dropdown menu set to '<Latest>'. A note below the dropdown states: 'Choose which server version to download. <Latest> gets the newest stable release.' There is also a checkbox for 'Full debug output' with the text 'Enable detailed server trace logging (see Output→RocketRide: Console)' below it. At the bottom, there is a 'Server Arguments' text input field with a placeholder text '--option=value --flag' and a note 'Additional command-line arguments passed to the server' below it.

Option 2: RocketRide Cloud

Pick **RocketRide Cloud** in the same dropdown, then:

1. **Sign in**. A green tick and your name confirm it

2. **Choose your Team**, which decides which team's engine you connect to
3. **Save All Settings**, top right. **It does not apply until you do**

That is it. The identical pipeline now runs on Cloud instead of your laptop.

The screenshot shows the 'Development Mode' settings dialog in RocketRide. At the top, there are tabs for 'Development' (selected), 'Deployment', 'Pipeline', 'Debugging', and 'Integrations'. The dialog has a title bar with 'Development Mode' and buttons for 'Cancel' and 'Save All Settings'. Below the title bar, a note states: 'Where pipelines run during development. Cloud and Direct Connect modes require authentication.' The 'Connection mode' section has a dropdown menu currently set to 'RocketRide Cloud'. Below this, a note says 'Choose where your server runs for development and deployment'. The 'Team' section features the RocketRide logo, a sign-in prompt, a green checkmark next to the username 'mithilesh gaurihar', a 'Sign Out' button, and a dropdown menu set to 'Development'. A final note at the bottom says 'Which team's engine to connect to'.

Build on Local, switch to Cloud for the demo. A judge opening a live link beats you screen sharing localhost.

If you type a Cloud address by hand it must be `https://` or `wss://`. Write `http://` or `ws://` and the connection **silently downgrades**, with no error. This applies to Cloud only. `ws://localhost:5565` for a local engine is correct.

6. Running from your own code

The moment your project has a UI or a script, you need the SDK.

```
Shell
pip install rocketride python-dotenv
```

Python

```
import asyncio, os
from dotenv import load_dotenv
from rocketride import RocketRideClient

load_dotenv()

async def main():
    async with RocketRideClient(
        uri=os.environ['ROCKETRIDE_URI'],
        auth=os.environ['ROCKETRIDE_APIKEY'],
    ) as client:
        result = await client.use(filepath='hello.pipe')
        token = result['token']
        try:
            out = await client.send(token, 'Hello, pipeline!',
                objinfo={'name': 'input.txt'},
                mimetype='text/plain')
            print(out)
        finally:
            await client.terminate(token)

asyncio.run(main())
```

Four calls: connect, `use()` to start and get a token, `send()` to push a payload, `terminate()` to stop. To run against Cloud, **change nothing in the code**, point `.env` at Cloud instead. Use `client.pipe()` for streaming large data.

terminate is not optional. The `async with` block calls `disconnect()`, but that **only drops the WebSocket, the pipeline keeps running on the engine**. Leave it out and every run leaves a live pipeline behind. We hit this internally and ended up with four orphans nobody could account for.

[TypeScript SDK](#) has the same shape.

§8. How to submit

The submission form is the submission

One per team, editable until the deadline. **Then post in #showcase on Discord** as well, which is not the submission but is how the community sees your work and how projects get picked up for `awesome-rocketride`. The form asks for that link.

You need: **a public GitHub repo with all your .pipe files in it**, a demo link or video, an app link if deployed, and slides if you made any.

Built in the Cloud Pipeline Builder? Download your `.pipe` files and commit them. This is the most common way a good project loses marks.

Before you post:

9. Runs from a clean clone
10. Secrets in `.env`, gitignored, every variable named `ROCKETRIDE_*`, plus a `.env.example`
11. README says what it does and how to run it, in five lines
12. **A batch run:** record count, cost, wall-clock time
13. Handles bad input without crashing, and escalates what it will not guess on
14. You know roughly what one run costs. The live trace shows token usage; "runs on Ollama, no API cost" is a valid answer

A live link beats a video. A video beats a promise.

9. Links

Discord	https://discord.gg/PMXrtenMsY
Cloud	https://cloud.rocketride.ai
Docs	https://docs.rocketride.org
Node reference	https://docs.rocketride.org/nodes
Pipeline JSON reference	https://docs.rocketride.org/pipeline-reference
Python SDK	https://docs.rocketride.org/develop/python
Troubleshooting	https://docs.rocketride.org/troubleshooting
VS Code extension	https://marketplace.visualstudio.com/items?itemName=RocketRide.rocketride
Open VSX, for Cursor and Windsurf	https://open-vsx.org/extension/RocketRide/rocketride
Community projects	https://github.com/rocketride-org/awesome-rocketride
Quickstart video	https://www.youtube.com/watch?v=bpoNvb8oOxY

Follow RocketRide: [LinkedIn](#) · [Instagram](#) · [X](#) · [Discord](#)

Setup checklist

Do this before August 31. One week goes fast, do not burn the first day on account setup.

- ☐ Join the Discord: <https://discord.gg/PMXrtenMsY>
- ☐ Create a RocketRide Cloud account: <https://cloud.rocketride.ai>
- ☐ Install the RocketRide extension in VS Code, Cursor or Windsurf
- ☐ Connect in Local mode and let the engine download. Free, no code needed
- ☐ **Build and run one small pipeline so you have seen it work**

Sort out a model: your own OpenAI or Anthropic key, or install Ollama and run one locally for free